Isaac's Tournament: A Comparison of The State of the Art in Multi-Robot Pursuit-Evasion Games

Luka Antonyshyn, Anthony Beca, Aoran Jiao, and Elliot Preston-Krebs
Institute for Aerospace Studies, University of Toronto, Toronto, Canada
{luka.antonyshyn, anthony.beca, aoran.jiao, elliot.prestonkrebs}@robotics.utias.utoronto.ca

***Abstract*—The study of planning under adversarial conditions has become of great interest as the adoption of robotics for security, defense, and surveillance increases. Further, adversarial planning can be used to account for worst-case scenarios in safety-critical areas, such as risk-aware motion planning or search and rescue. Often, this study is conducted under the framework of differential game theory, a methodology that studies optimal outcomes of scenarios with multiple rational actors that evolve according to differential equations. One such scenario is the N-pursuer visibility-based pursuit evasion game, in which a team of pursuers attempts to examine an entire area in such a way that a moving evader would be unable to escape detection. This game has been used to model environmental inspection, perimeter defense, and search and rescue problems, but most approaches tend to only evaluate simple scenarios and compare against similar approaches. In this report, we rectify this gap through the study and comparison of the quantitative and qualitative results associated with four state of the art approaches.**

I. Introduction

Adversarial games in continuous spaces have long been studied under the field of differential game theory, developed by Rufus Isaac's in the 1960s [1]. These games, contrary to their discrete classical game theoretical counterpoints, have the evolution of state evolve according to differential equations. Differential games have been used in several application spaces, such as perimeter defense, risk-averse landing path planning for airplanes, and search and rescue [2].

Recently, more interest has been taken in multi-agent differential games. These, of course, model more realistic scenarios and engagements. However, solution methods must often settle for sub-optimal solutions due to the curse of dimensionality and the computational difficulty of obtaining analytical solutions [3]. As a result, much work has been focussed on the development of heuristics, learning-based approaches, and approximate solutions.

One such multi-agent game is the multi-pursuer vision-based pursuit evasion game. In this formulation, the evaders are often treated as abstract entities that can move at arbitrary speeds. This forces the pursuing team to plan for the worst-case scenario. As such, the problem devolves into a type of constrained frontier expansion problem on a known environment. This makes the formulation of interest to motion planning researchers. A depiction of such a game, along with a valid solution for clearing the map, is depicted in Figure 1.

Many solutions have been proposed for this problem, from learning-based methods [4], sampling-based approaches [5], [6], and graph-search algorithms [7]. However, in the literature, practitioners tend to compare only against solutions they have readily available - ignoring suitable baselines from other problem formulations and only evaluating on limited subsets of the total problem space. As such, the state of the art is lacking in honest analysis and comparison of different approaches to multi-agent differential games.

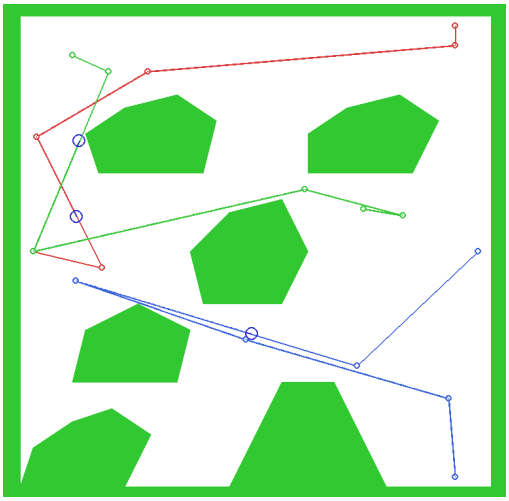


Fig. 1: A sample map (obstacles and boundaries in green), with three agents (medium blue circles) traversing their valid motion plans (coloured lines and nodes) through a joint configuration space.

Following the growth of popularity of game theory, an effort to evaluate the performance of different strategies was undertaken in the form of Axelrod's tournament - a contest in which different policies are pitted against each other in varying conditions to compare outcomes. Here, we pursue a simplified version of Axelrod's tournament on one differential game. We implement four approaches to the multi-pursuer observability-based pursuit evasion game, and compare them on numerous scenarios using common metrics. We identify and analyse the behaviours of each method under various conditions that challenge their formulations, and use the information to recommend steps for future works.

The remainder of this work is organized as follows; in Section II, we state the exact problem formulation and review necessary background information; in Section III, we review the related works; in Section IV, we detail the individual

algorithms we implement and evaluate; in Section V, we detail our methodology for comparison; in Section VI, we discuss and analyse the methods comparisons and failure modes; and in Section VII, we identify promising areas for further investigation.

II. Background

A. Pursuit-Evasion Games

Pursuit-evasion games study the interaction between one or more pursuers and one or more evaders moving in a configuration space $\mathcal{D} \subset \mathbb{R}^d$. The objective of the pursuers is to capture the evaders, while the evaders aim to avoid capture. These problems arise in robotics, control theory, and differential games [1].

A common formulation models the players as dynamical systems. For a single pursuer with state $x_p(t)$ and evader with state $x_e(t)$, their dynamics may be written as

$$\dot{x}_p(t) = u_p(t), \quad \dot{x}_e(t) = u_e(t),$$

where $u_p(t), u_e(t) \in \mathcal{U}$ are control inputs subject to bounded speed constraints. The game terminates upon capture, defined as

$$\|x_p(t) - x_e(t)\| \leq \varepsilon,$$

for some radius $\varepsilon > 0$.

The pursuer utilizes a control strategy $u_p(\cdot)$ that guarantees capture in finite time, while the evader uses a strategy $u_e(\cdot)$ to maximize the capture time. Optimal strategies are usually recovered using methods from differential game theory, such as Hamilton-Jacobi-Isaacs equations [1]. In higher dimensional spaces, or even with more complex kinematics, solutions to these equations are often intractable to compute.

B. The N-Pursuer Visibility-Based Pursuit-Evasion Game

In the N-pursuer visibility-based pursuit-evasion problem, a team of N pursuers cooperatively searches for an evader with an unknown position in a known environment $\mathcal{D}$. We denote the configuration space of the environment, exclusive of obstacles, as $\mathcal{C} \subset \mathcal{D}$. One key distinction from the standard model of the pursuit-evasion game is that information is limited by line-of-sight visibility: the evader is detected only if it lies within the visible region of at least one pursuer. This visible region is defined by the joint configurations of the pursuer, the shape of $\mathcal{C}$, and the position of obstacles [8].

Let the pursuers have states $\{x_i(t)\}_{i=1:N} \subset \mathcal{C}$. The locations of obstacles in configuration space define a visibility map. For a pursuer at x_i, its visibility region is

$$\mathcal{V}(x_i) = \{y \in \mathcal{D} \mid y \text{ is visible from } x_i\}.$$

We refer to the joint Visibility region of all pursuers as $\mathcal{V}$. In this variation of the pursuit evasion-game, the evader is often treated as an abstraction rather than a physical entity. We assume the position of the pursuer $e(t) \in \mathcal{C}$ at time t is a continuous function $e : [0, \infty) \to \mathcal{C}$ that describes the path of the evader moving at arbitrary speeds. This allows the evader to inhabit any location in a connected subspace $\mathcal{S} \in \frac{\mathcal{C}}{\mathcal{V}(x)}$, with the fraction denoting the exclusion of $\mathcal{V}(x)$ from $\mathcal{C}$. We refer to $\mathcal{S}(t)$ as the shadow set at time t.

Unlike classical pursuit-evasion, the goal is not immediate capture of the evader but guaranteed detection. The pursuers seek strategies that ensure that, regardless of the evader trajectory, the evader will eventually be observed:

$$\exists\, T < \infty \text{ such that } e(T) \in \mathcal{V}()T).$$

A central concept in these games is the partition of $\mathcal{C}$ into cleared and contaminated regions. A point $y \in \mathcal{C}$ is cleared at time t if it has been visible to some pursuer at some prior time and cannot be re-entered by the evader without being detected. This is equivalent to it not having been path-connected to a contaminated region. Otherwise, the point is contaminated. Let $\tilde{\mathcal{S}}(t)$ and $\bar{\mathcal{S}}(t)$ denote the cleared and contaminated regions, respectively, with

$$\mathcal{C} = \tilde{\mathcal{S}}(t) \cup \bar{\mathcal{S}}(t).$$

Pursuer strategies must ensure that $\bar{\mathcal{S}}(t)$ shrinks monotonically, avoiding recontamination. This effectively collapses to a constrained form of the frontier expansion problem, where frontiers are dynamic and may rejoin each other.

The visibility-based pursuit-evasion problem is significantly harder than classical pursuit-evasion due to partial information and the need for coordination. This leads to many discretizations, sampling-based methods and learning approaches to the problem.

C. Shadow Events

Central to some algorithms in the literature is the concept of shadow events. This framework provides a conceptual analog of pursuer movement that neatly fits into the N-Pursuer Visibility-Based Pursuit-Evasion game. Briefly, shadow events describe a framework for propagating labels assigned to particular unseen regions in a space based on a hierarchy of dominance under common interactions.

Introduced in [9], shadow events are primarily used to keep track of contaminated areas in the game's space of interest $\mathcal{D}$. As such, the labels of concern are whether an area is clear or contaminated. Naturally, contaminated labels dominate cleared labels, as when a shadow we have cleared intersects with an unchecked shadow, the evading agent may move into the previously checked space $\tilde{\mathcal{S}}$.

Five main shadow events are introduced. These are

- Appear - A shadow is instantiated when a previously seen part of the environment becomes hidden.
- Disappear - Existing shadows vanish when they come into the visibile set of the pursuers $\mathcal{V}$.
- Merge - Two or more existing shadows join together and adopt the same shadow label when a shadow appears linking them together, based on dominance.
- Split - One shadow can be divided into multiple shadows when the pursuers visibility cones cause them to be non-path connected in 4-neighbour space.

In practice these events are calculated on a continuous space by discretizing into a grid, calculating shadows based on line

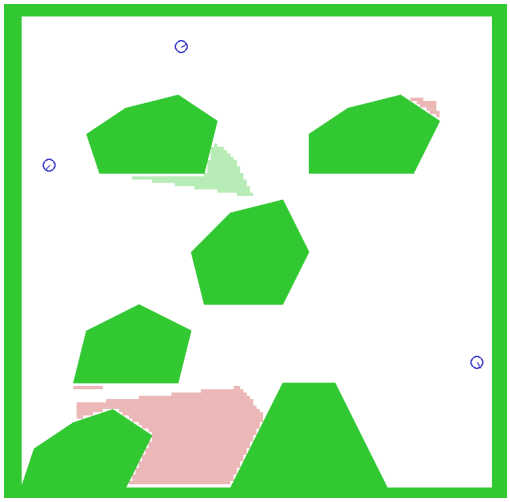


Fig. 2: Cleared (green) and contaminated (red) shadows. The status of the shadow labels are determined by the history of the game and the existence and configuration of the shadows are determined by the line of sight of the pursuers (blue).

of sight from agent positions in a joint configuration, and propagating dominant labels to the 4-neighbours of any unseen cell.

We may now state the relationship between the shadow events and labels as

- Appear - A shadow appearing from previously visible areas $\mathcal{V}(t)$ is marked clear.
- Disappear - All disappearing labels are discarded.
- Merge - When shadows merge, the dominant label is assigned to all of the shadows. That is to say, if any of the merging shadows is contaminated,the resulting shadow is contaminated, and otherwise it is clear.
- Split - All shadows resulting from a split inherit the label of the original shadow.

Of course, algorithms may use this information in their planning loop, and several considered implementations do so [5]. The primary drawback of this approach is the extreme cost of computing the shadow label propagation. in a 64mx64m environment at a resolution of 1m, discretization yields 4096 cells, each of which has to be check for line of sight from N positions. Every resulting shadow position then must be checked for neighbours, resulting in a breadth first search through the graph for all connected shadows. This leads to significant slow-downs in sampling on more complex and larger maps. Speed-ups can be made by decreasing the resolution, but this comes at the cost of potentially missing contaminations. We provide a visualization of these shadows in the midst of a plan being executed in Figure 2.

Nonetheless, this concept provides a simple way to monitor and evaluate approaches to this problem. As such we find it useful to utilize shadow labels as a metric in the evaluation of the various algorithms detailed below.

D. Probabilistic Road Maps

The Probabilistic Roadmap (PRM) is a sampling-based motion planning algorithm designed for multi-query scenarios in high-dimensional configuration spaces [10]. PRM constructs an undirected graph (roadmap) $G = (V, E)$ that approximates the connectivity of $\mathcal{C}$. The algorithm proceeds in two phases; a sampling phase and a query phase.

During the initial sampling phase, a set of n configurations $\{q_1, \ldots, q_n\}$ is sampled uniformly from $\mathcal{C}$

$$q_i \sim \mathrm{Unif}(\mathcal{C}).$$

Each valid sample becomes a vertex in V. For each q_i, a set of neighboring vertices is selected using a metric $\rho(\cdot, \cdot)$. For each neighbor q_j, a local planner attempts to connect q_i and q_j by a simple path:

$$\epsilon_{ij}(t) = (1 - t)q_i + tq_j, \quad t.$$

An edge $(q_i, q_j) \in E$ is added if $\epsilon_{ij}(t) \subset \mathcal{C}$.

In the query phase, given start and goal configurations $q_{\text{start}}, q_{\text{goal}} \in \mathcal{C}$, are connected to nearby vertices in G using the local planner. A graph search algorithm is then used to find a path in G.

PRM is probabilistically complete: if a path exists in $\mathcal{C}$, the probability that PRM finds a solution approaches 1 as $n \to \infty$. Under appropriate connection strategies, PRM variants can achieve asymptotic optimality, and have been applied to numerous multi-robot problems [11].

E. Graph-Based Motion Planning

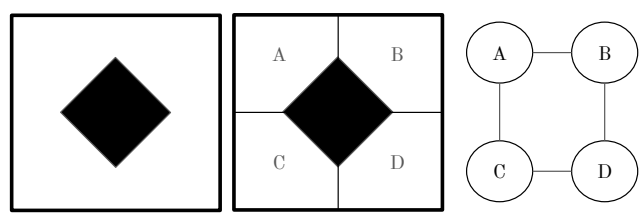


Fig. 3: Graph construcion from simple map: (1) polygonal environment, (2) convex decomposition, (3) topological representation.

Graph-based motion planning transforms the continuous C-space into a discrete topological representation $G = (V, E)$. In this formulation, vertices $v \in V$ represent discrete configurations in $\mathcal{C}$, while edges $\epsilon \in E$ represent feasible transitions between them. The framework typically follows a three step pipeline: a discretization phase, followed by edge weight assignment based on cost metrics, and a graph search algorithm to extract a least-cost path.

The literature offers diverse approaches. Discretization may utilize exact cell decompositions (i.e. visibility graphs, Morse cells, or convex partitions [12]), or approximate grid-based decompositions. While classic methods like Breadth-First-Search (BFS) or Depth-First-Search (DFS) provide basic connectivity, modern motion planning heavily leverages informed

search heuristics (namely A*) to manage the computational complexity inherent in high-dimensional state spaces.

F. Multi-Agent Reinforcement Learning

Reinforcement learning (RL) studies sequential decision-making problems in which an agent selects actions from observations and updates its policy using scalar rewards. In the standard Markov decision process (MDP) formulation, a policy $\pi_\theta(a_t \mid s_t)$ is trained to maximize discounted expected return,

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right].$$

Robotic pursuit-evasion is usually only partially observable, so the policy is often conditioned on local observations o_t, partial maps, or observation histories rather than the true full state.

Multi-agent reinforcement learning (MARL) generalizes this setting to N decision-making agents [13]. Each agent i receives an observation $o_i(t)$, chooses an action $a_i(t)$, and the joint action $a(t) = (a_1(t), \ldots, a_N(t))$ determines the next state and team reward. In cooperative games, the agents may share a common reward, but the policy must still determine which individual actions contributed to the team-level outcome. This is the credit assignment problem. Independent learning also makes the environment non-stationary from the perspective of any one agent, since the other agents' policies change during training.

A common way to address this issue is centralized training with decentralized execution (CTDE) [14]. During training, a critic or value estimator may use additional information such as the full team state, while execution is performed by decentralized policies. Actor-critic algorithms are a common implementation of this idea: the actor represents the policy, while the critic estimates the value of states or state-action pairs and guides the policy update. Off-policy methods reuse stored experience, while on-policy methods update from recently collected trajectories. For spatial multi-agent problems, learned policies are often paired with graph neural networks or attention layers so that variable-size maps and teams can be encoded without flattening everything into a fixed-size vector.

III. Literature Review

The pursuit-evasion game was originally introduced by Rufus Isaacs in his seminal work on differential game theory [1]. Isaacs detailed a method for finding optimal solutions to differential games. The solution to this equation is tractable to calculate analytically in low-dimensional spaces with simple dynamics. As we are concerned with a far more complex game with high dimensionality, visibility constraints, and more complex configuration spaces, we omit discussion of analytical solutions and instead focus on approximate methods. Below, we provide an overview of several approaches to the N-pursuer visibility-based pursuit-evasion game.

A. Sampling-based Approaches

Sampling-based motion planning methods have long been studied for their ability to find high-quality solutions to challenging configuration spaces in reasonable amounts of time [15]. Common sampling-based methods include the randomly-exploring rapid tree algorithms [16], the fast marching tree algorithm [15], and the probabilistic road-map (PRM) algorithm [10]. Unfortunately, these algorithms tend to struggle in joint configuration spaces as a result of the curse of dimensionality, leading to several multi-robot variations [11].

Due to the complex nature of the problem space and visibilty model, sampling-based approaches to the this game are fairly common. Most works are in this space utilize a PRM-style planner. In one such method, Stifler and O'Kane propose on such planner that utilizes an internal model of the evolution of visible spaces as an additional validity check on expanding nodes during planning [5]. In a follow up work, the algorithm performance is improved through the use of domain-specific pose sampling, and a solution-refinement method [17]. Several additional extensions to this method have been considered, including the improvement of robustness to individual robot failures [18], the addition of probabilistic completeness guarantees [19], and the consideration of more complex sensing models [6].

B. Graph-based Approaches

Guibas and Lavalle [9] formalized the pursuit-evasion problem in continuous polygonal environments with recontaminations. They develop a complete algorithm for the case of a single pursuer. First, the workspace is partititioned into convex regions, with binary labels indicating contamination, then a search is performed on the resulting quotient information space. By the definition of convexity, all points in a convex cell are mutually visible [20], therefore, the pursuer's progress is tracked by its transitions between these cells. The distance between centroids as well as the contamination state are used as an information-based heuristic for Dijkstra's algorithm. They show a logarithmic bound on the number of needed pursuers (i.e. the search number) for simply-connected free spaces, but leave multi-agent algorithms for future work. As part of this work, we implemented the Guibas algorithm for a single pursuer.

The most relevant paper for our tournament (respecting all game constraints) proposes Graph-Clear [7]. A team of robots can execute *sweep* actions to detect evaders in a vertex, and *block* actions to prevent intruders from crossing an edge. A strategy consists of a sequence of block and sweep actions to detect all intruders despite possible recontamination. They show the computation of optimal strategies for this game is NP-hard for graphs; therefore the authors propose to create spanning trees, for which a Graph-Clear is a polynomial time algorithm. Intuitively, cycles in the graph are broken by agents executing a block action, reducing the problem to a tree search to be executed by the remaining agents.

Some recent works tackle similar problem formulations, but do not qualify for the restrictions of this comparison. Namely,

Zhou [21] and Huang [22] separately develop decentralized strategies for multiple pursuers in planar environments based on Voronoi decompositions; but evaders have finite speed. Bhattacharya et. al [23] analyze a cops and robber game in multidimensional grids, but cops are able to "teleport" across cells.

C. Learning-based Approaches

Learning-based approaches to pursuit-evasion can be separated into two broad groups. The first considers pursuit as a chasing or interception problem, where the evader's state is available to the pursuers. For example, decentralized deep RL has been used to train multiple pursuers under local sensing and communication constraints [24]. These methods are relevant for multi-agent coordination, but they do not address the clearing problem considered here because the evader is localized and has bounded dynamics.

The second group uses learning as a substitute for explicit search over the information space. Rather than enumerating all possible contamination labels, a policy learns useful clearing and guarding behavior from simulation. This can scale better at execution time, but it trades away the completeness guarantees associated with graph-search methods.

ViPER (Visibility-based Pursuit-Evasion via Reinforcement Learning) is the most directly relevant learning-based approach for this report [4]. It is designed specifically for visibility-based clearing with omniscient, potentially arbitrarily fast evaders. ViPER builds a graph of candidate viewpoints over the known free space, augments each node with frontier and clearing information, and uses graph attention to learn which neighboring viewpoint each agent should visit next. The method follows CTDE, similar in spirit to multi-agent actor-critic methods such as MADDPG [14]: the deployed policy is decentralized, while the training critic uses privileged map and team-action information. ViPER also uses an attentive critic, similar in motivation to MAAC [25], to improve credit assignment. Its main limitation is that it does not certify clearing in the same way as complete planners; success must be measured by the simulator or an external contamination tracker.

IV. Implementation

As we are interested in comparing recent developments in the field, we sampled and implemented 4 algorithms, with at least one from each problem formulation. These are then compared on common metrics using a series of different maps. Below, we detail every algorithm selected for comparison.

A. Sample-Generated Pursuit-Evasion Graphs

Sample-generated pursuit-evasion graphs (SG-PEG) utilize PRM-style sampling in a joint configuration space to find solutions to the visibility-base pursuit-evasion game. While based on PRM algorihtms, no proofs of probabilistic completeness are provided for SG-PEG [5].

We begin with a description of the graph data structure utilized in SG-PEG. The graph utilized is a rooted di-graph

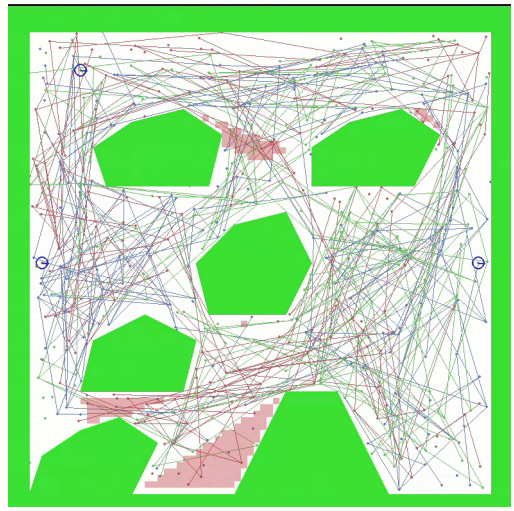


Fig. 4: SG-PEG after two rounds of sampling and edge connections

with vertices representing joint robot configurations $p_{0:N}$, as well as the set of non-dominated shadows reachable by traversing the graph to the vertex in question. For an edge to be considered valid in the context of the graph, a line in the joint configuration space $\mathcal{C}^N$ must exist that maps reachable configurations of shadow labels at the source vertex to the same at the target vertex.

The core complexity in the graph arises from the operation necessary to add an edge. When a new connection is established between two vertices, resulting shadow labels must be computed. To do so, interpolation between the two associated joint configurations is utilized to obtain a number of evenly spaced poses in $\mathcal{C}^N$. Shadow events are then computed at these evenly spaced joint configurations, propagating the intermediate labels between the vertices.

Upon adding a new edge, the algorithm recursively searches through the neighbours of the vertex, updating the dominated labels. This acts as bookkeeping for finding a solution. If the reachable labels are all clear for a given vertex, the vertex must denote a full clearing of the environment. Upon this condition being met, we may then reconstruct a solution from the intermediate waypoints between the root and the cleared node, as in any sampling-based motion planning method. A visualization of the sampling process after a number of iterations can be seen in Figure 4.

The authors utilize three constraints to filter which edges should be added to the graph. The first is mutual visibility - a line of sight, or obstacle free straight line path, must exist between each pursuers configuration at vertex i and its corresponding configuration at vertex $i+1$. The second is edge-length, with a maximum distance being set between two configurations. Finally, a minimum cycle length is set to ensure that computation is not wasted on low-information configuration changes. Numerous sampling strategies are explored, but none seem to outperform uniform sampling in the joint configuration space. The full procedure is outlined in pseudo-

code in Algorithm 1.

Algorithm 1 Sampling-Based Pursuit-Evasion (SB-PEG)

Require: Configuration Space $\mathcal{C}$, N pursuers, initial joint state q_0

Ensure: Joint pursuer paths achieving full visibility coverage

Initialize graph $\mathcal{G}$ with root node q_0
Initialize visibility coverage $\mathcal{V}$
while there exists unseen region in $\mathcal{C}$ **do**
    Sample a random joint configuration $p_{\text{rand}} \in \mathcal{C}$
    Select nearest nodes $v_{v:j} \in \mathcal{T}$ to p_{rand}
    Generate a feasible motions σ_i for pursuers from $q_{v:j}$
    Construct new joint state q'
    Compute updated visibility:

$$\mathcal{V} \leftarrow \mathcal{V} \cup V(q')$$

    if $\tilde{\mathcal{S}}(q')$ strictly increases **then**
        Add q' to $\mathcal{G}$ with edges $(q_{v:j}, q')$
    end if
end while
Extract a subtree $\mathcal{G}^*$ achieving full coverage
return joint pursuer trajectories

B. Visibility Roadmaps

In [17], an extension to SG-PEG is proposed. Several key innovations are detailed, including an improved visibility-based sampling distribution, a solution refinement method, and a process for iteratively increasing the number of pursuers until a solution can be provided [17]. Due to the references to visibility roadmaps in the paper title, we refer to this variant as visibility-roadmap SG-PEG (VR-SG-PEG). We note that, as we are interested in evaluating each detailed algorithm in sterile and identical conditions, we omit the process of increasing the number of pursuers from our method, instead keeping this fixed based on a given configuration.

In place of the uniform sampling recommended in SG-PEG, VR-SG-PEG uses an informed sampling method, referred to as web-sampling, to improve the density of samples in interesting regions. By extension, this nominally improves convergence properties of the planner, at the cost of higher levels of computation. The sampling proceeds in two phases: an initial visibility covering phase and an intersection sampling phase.

In the first phase, a set of initial points $\mathcal{P} \subset \mathcal{C}$ is constructed, referred to as a covering set. This set is generated as follows: each initial point p is sampled sequentially and randomly from outside the set of visible points $\mathcal{V}$, i.e a sample p_i is selected such that $p_i \notin \cup_{p_j \in \mathcal{P}} \mathcal{V}(p_j)$. This process continues until these mutually non-visible points cover the entire configuration space,$\cup_{p_j \in \mathcal{P}} \mathcal{V}(p_j) = \mathcal{C}$.

In the second phase, a series of points, denoted by the set $\mathcal{Q}$, are sampled to expand the roadmap data structure. This is done by sampling pairs of points $p_i, p_j \in \mathcal{P}$, denoted intersection points. A point is then sampled uniformly from the intersection of the individual visibility cones of each point $\mathcal{V}(p_i) \cap \mathcal{V}(p_j)$, if the intersection is non-empty. The so-called web is the union of the initial and intersection points $\mathcal{W} = \mathcal{P} \cup \mathcal{Q}$. A separate web is sampled for each pursuer, and if a web is ever exhausted, a new set of webs is generated to continue. The pursuit-evasion graph is constructed by sampling points from these webs without replacement. A visualization of the sampled and connected graph can be seen in Figure 5.

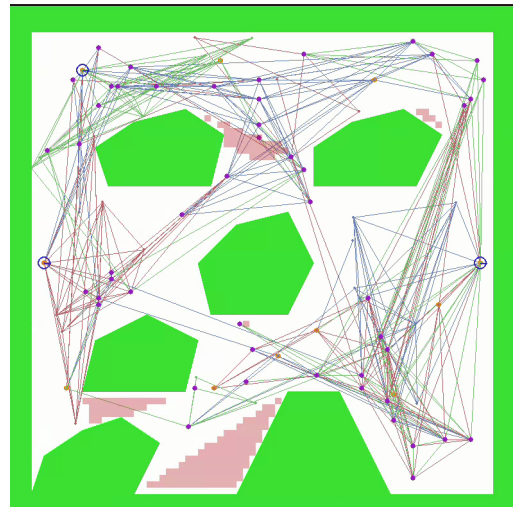


Fig. 5: VR-SG-PEG after two rounds of sampling and edge connections

The second innovation of note is the inclusion of a solution refinement step, devised to reduce unnecessary movement of the robots. This acts as a post-processing step, filtering out the extraneous plan branches. Iteratively, two non-adjacent points from the solution z_a, z_b are selected to check if a shortcut from z_a to z_b is still a valid solution. In addition to standard kinematic and collision avoidance constraints, the algorithm checks whether adjusting the solution in this manner clears the same set of shadows as the original solution, using the procedure laid out in Subsection IV-A.

We lay out the entire procedure in Algorithm **??**.

C. Pursuit Evasion on Trees

The Graph-Clear algorithm [7] provides a complete, polynomial-time clearing strategy on a surveillance graph. Agents have two actions, *block* and *sweep*, and work together to reduce graphs to spanning trees that can be searched in polynomial time. In the first stage of the algorithm, C_{free} is decomposed into convex cells, per Fig. 6, with vertices representing the cells, and edges representing the boundaries between cells. An important property of convex cells is mutual visibility of all points within the set; an agent on the boundary of two cells can surveil both.

The state space of a surveillance graph G is defined to be the set

$$V(G) = \{R, C\}^n \times \{R, C, B\}^m$$

where R represents `clear`, C represents `contaminated`, and B represents `blocked`, n is the number of vertices, and m the number of edges. A *recontamination path* is defined in

Algorithm 2 Visibility Roadmap Sampling for Multi-Pursuer Pursuit-Evasion

Require: Configuration Space $\mathcal{C}$, N pursuers, initial joint state q_0

Ensure: Joint pursuer paths achieving full visibility coverage

Initialize roadmap $\mathcal{G} \leftarrow (\mathcal{V}, \mathcal{E})$ with root q_0
Initialize visibility coverage $\mathcal{V}(q_0)$
Initialize priority queue $\mathcal{Q} \leftarrow \{q_0\}$
while coverage not complete **do**
    Sample a pursuer index $i \in \{1, \ldots, N\}$
    Sample a path σ_i for pursuer i using web sampling
    Compute new visibility region $\mathcal{V}_{new} \leftarrow \mathcal{V} \cup \mathcal{V}(\sigma)$
    if $\mathcal{V}_{new}$ adds new area **then**
        Add new node q' to $\tilde{\mathcal{S}}$ with coverage $\mathcal{V}_{new}$
        Add edge (q, q') to $\mathcal{E}$
        Insert q' into $\mathcal{Q}$ (prioritized by coverage gain)
    end if
end while
Extract solution subtree $\mathcal{T} \subseteq \mathcal{G}$ achieving full coverage
// Refinement Phase
for each path segment in $\mathcal{T}$ **do**
    Locally resample and shortcut paths to reduce cost
    Preserve visibility constraints during refinement
end for
return joint pursuer trajectories from $\mathcal{T}$

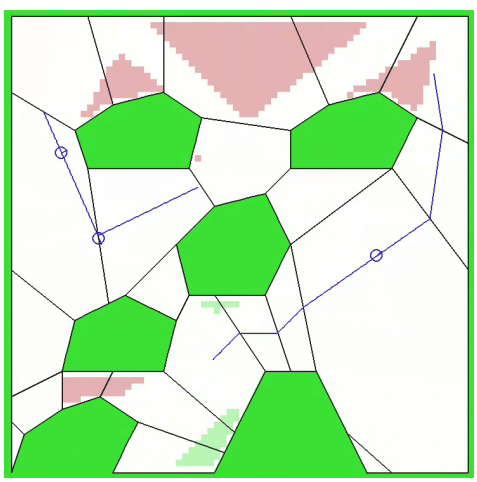


Fig. 6: Graph-clear on the `convex_obstacles` map. Note the convex cell decomposition of the configuration space.

the topological space, denoting a path of edges and certices on which no edge is blocked.

The two actions that can be taken by agents are *sweeping* and *blocking*. Sweeping actions enter a convex cell, and mark the associated vertex clear in the surveillance graph. Blocking actions entail agents waiting on the boundary of two cells, breaking cycles in the topology such that a subgraph is a spanning tree for other agents to decontaminate. Agents work together to coordinate

The algorithm processes the surveillance tree bottom-up, associating an optimal *full-cut sequence* $\bar{S}_{v_x}(e)$ with each directed edge $e = [v_x, v_y]$, representing the cheapest contiguous progressive clearing order for the subtree rooted at v_y once v_x is already clear. The algorithm is shown below:

Algorithm 3 Graph-Clear: Optimal Contiguous Strategy on a Tree

procedure GRAPHCLEAR($T = (V, E, w)$)
    Initialize $\bar{S}_v(e) \leftarrow \emptyset$ for every directed edge e
    $Q \leftarrow$ all leaves of T
    while Q is not empty **do**
        $v_y \leftarrow Q$.dequeue()
        if v_y is a leaf **then**
            $v_x \leftarrow$ unique neighbor of v_y
            $\bar{S}_{v_x}([v_x, v_y]) \leftarrow \{\{v_x\}, \{v_x, v_y\}\}$
        else
            Let v_x, v_y be neighbours; $\bar{S}_{v_x}([v_x, v_y]) = \emptyset$
            Let $v_1, \ldots, v_k$ be the remaining neighbors
            Merge $\bar{S}_{v_y}([v_y, v_i])$ for $i = 1, \ldots, k$, ordered by ρ increasing, to form the optimal strategy S_{v_y} on the subtree $T[v_y]$
            Prepend v_x to each cut of S_{v_y} and filter, yielding $\bar{S}_{v_x}([v_x, v_y])$
            if exactly one neighbor of v_x still has no computed sequence **then**
                Q.enqueue(v_x)
            else if all neighbors of v_x have computed sequences **then**
                Q.enqueue(v_x)
            end if
        end if
    end while
    for each $v \in V$ **do**
        Assemble S_v by merging all $\bar{S}_v([v, u])$ for neighbors u, ordered by ρ
        $ag(v) \leftarrow \text{cost}(S_v)$
    end for
    return $\min_{v \in V} \; ag(v)$
end procedure

D. ViPER

ViPER is the learning-based algorithm included in our comparison. It represents the known free space as a graph $G_t = (V_t, E_t)$ of candidate viewpoints and traversable edges. At every decision step, each pursuer observes an augmented version of this graph containing relative node positions, teammate occupancy, frontier utility, uncovered-frontier utility, cleared-state indicators, and an A^* guide signal. The shared policy $\pi_\theta(a_i(t) \mid o_i(t))$ then chooses a neighboring viewpoint for each pursuer, so the learned action space remains tied to feasible motion on the map.

The map is divided into occupied cells $\mathcal{M}_o$, free cells $\mathcal{M}_f$, and cleared cells $\mathcal{M}_c$; uncovered frontiers between $\mathcal{M}_c$ and $\mathcal{M}_f \setminus \mathcal{M}_c$ are the locations through which a worst-case evader can recontaminate cleared space. ViPER optimizes

synchronized team motion, where the cost of a step is the longest distance traveled by any pursuer before the next decision. Its shared reward is

$$r_t = c_1 \left(|\mathcal{M}_{c,t+1}| - |\mathcal{M}_{c,t}|\right) - c_2 \max_i \|\psi_i(t+1) - \psi_i(t)\| + \mathbf{1}_{\mathcal{M}_c = \mathcal{M}_f^*} r_f.$$

The first term rewards net clearing, the second penalizes team travel, and the final term rewards full task completion.

The network uses graph attention to encode the current viewpoint graph and a decoder that scores only feasible neighboring nodes. Training uses Soft Actor-Critic [26] under CTDE: the actor receives the decentralized graph observation, while the critic is given privileged map and team-action information. During evaluation, only the actor is used. In our implementation, ViPER is treated as an online planner: we construct the visibility graph, compute node features from the shared map, query the actor for each pursuer, execute the selected moves, and evaluate the resulting trajectory with the same shadow-tracking and tournament metrics used for the other methods.

A key node feature is the *safe_utility*: the number of safe-zone frontier cells visible from a given node within a fixed sensing range, accounting for line-of-sight. Nodes deep inside the already-cleared region have low safe_utility (few reachable frontiers), while nodes at the perimeter of the cleared zone, where new area can be uncovered, have high safe_utility. Because frontiers shift as agents advance, safe_utility is re-computed at every decision step; the policy is trained to steer agents toward high-utility nodes to maximise clearance rate. Figure 7 shows a snapshot of the viewpoint graph during execution: node color encodes safe_utility, ranging from dark purple (low) through teal to yellow (high). Agents are shown as open circles connected by a red line indicating their current heading direction, and the pink shaded regions are areas that have been recontaminated and must be re-cleared.

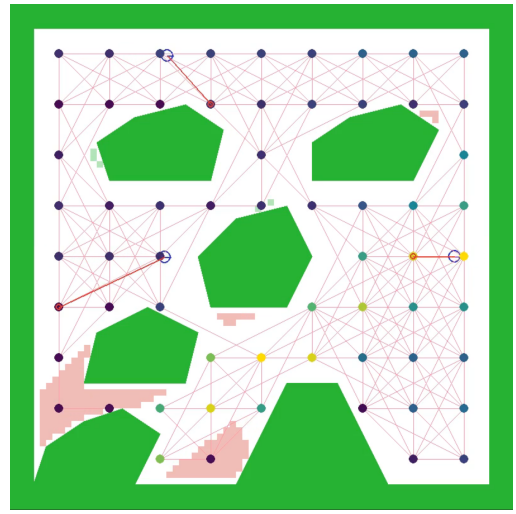


Fig. 7: Snapshot of the ViPER viewpoint graph during execution on a representative map. Node color encodes *safe_utility* — the number of safe-zone frontier cells visible from each node — from dark purple (low value, deep inside the cleared region) to light yellow (high value, at the active clearing frontier). Open circles mark the pursuer positions; the red line indicates the direction of current motion for each pursuer. Pink regions are recontaminated areas awaiting re-clearance. As clearing progresses, frontier nodes become interior nodes and their utility drops, driving agents onward to new frontiers.

V. Comparison Methodology

We evaluate the methods described in Section IV using a unified set of metrics: planning time, execution time, total path length, maximum environment coverage, and success rate. P ath length is computed as the cumulative distance traveled by all pursuers and is measured online during execution. Experiments are conducted on four maps with varying structural complexity: `simple_obs`, `ring`, `H_map`, and `convex_obstacles`. These environments range in size from $40 \times 40\,\mathrm{m}$ to $64 \times 64\,\mathrm{m}$ and contain both convex and non-convex obstacles. The `simple_obs` map includes several polygonal obstacles and a large circular obstacle, which complicates decomposition into convex regions for methods which incroperate this into planning, visualized in Figure 8a. The `ring` map consists of two enclosing walls with narrow gaps, requiring coordinated clearing to avoid recontamination, visualized in Figure 8b. The `H_map` contains an H-shaped structure with enclosed pockets that must be cleared edliberatly before advancing, visualized in Figure 8c. Finally, `convex_obstacles` features multiple convex obstacles and presents the greatest challenge for recontamiation, visualized in Figure 8d.

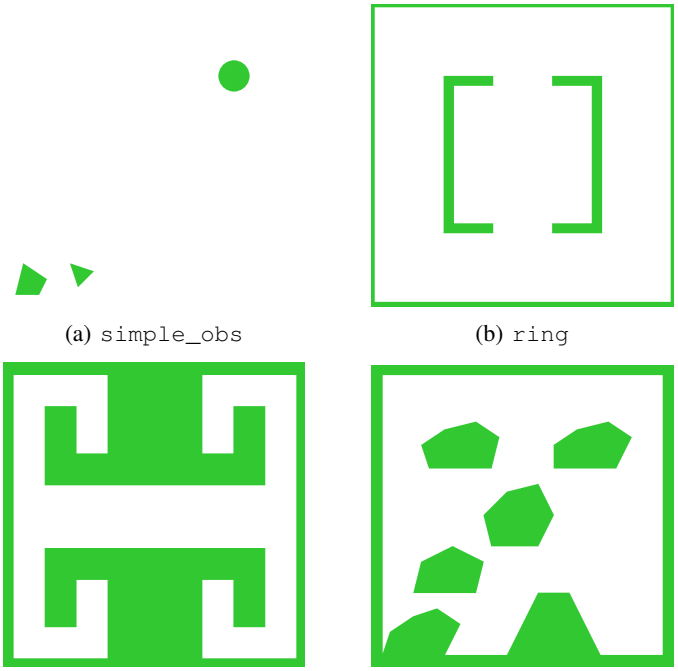
For each map, we generate 30 sets of random initial pursuer configurations. These spawn configurations are held constant across all methods to ensure fair comparison. In addition, we evaluate scalability by varying the number of pursuers on the `simple_obs` map.

Success is determined based on shadow clearance described in Subsection II-C. A trial is terminated early and marked successful if 100% coverage is achieved. Trials that reach at least 99% maximum coverage are also counted as successful post hoc.

To ensure consistent runtime constraints, we impose a global timeout of 3.3 minutes per trial, encompassing both planning and execution. Additionally, a 3 minute timeout is enforced for individual planning steps. If a planning timeout is reached, any available partial plan is executed for the remainder of the trial. This constraint primarily affects single-shot planners such as SG-PEG and VR-SG-PEG.

VI. Numerical Results

The results of our comparison are summarized in Table I and Table II. Table I compares the performance of the four algorithms across the different maps with a fixed number of pursuers (4), while Table II illustrates how performance varies as the number of pursuers increases on `simple_obs`.



(a) simple_obs (b) ring

(c) H_map (d) convex_obstacles

Fig. 8: Visualization of four maps used in evaluation.

A. Map Comparison

On environments with simpler obstacle arrangments (simple_obs and ring) every algorithm is generally capable of achieving high coverage, although with notable differences in reliability and efficiency.In particular, SG-PEG and VR-SG-PEG exhibit reduced success rates on ring (53.33% and 83.33%, respectively), despite maintaining high maximum coverage. This suggests that these methods struggled with coordinated clearance and often fail to prevent recontamiation. Graph-Clear acheives high coverage and success rates which demonstrate the effectiveness of its methodical approach to clearing and gaurding convex regions. However, we can observe on simple_obs, that the run time is significatly larger. This can be attributed to the difficulty of decomposing maps with circular obstacles into convex regions. Lastly, ViPER consistently achieves 100% success on these maps while maintaining competitive execution times and shorter path lengths, indicating more efficient coverage.

In terms of the more complex maps, the performance of these methods generally degrades, either in success or efficiency. In H_map, SG-PEG and VR-SG-PEG struggle significantly, with success rates 20%, while Graph-Clear and ViPER maintain very high success rates. On this map, we also see much larger planning and execution times on all algorithms but ViPER. In fact, the planning times of the sampling based planners suggest that the planning timeout was often triggered, causing partial paths to be executed, a likley culprit for the degraded coverage performance and very short average path length for VR-SG-PEG. We do also see that both Graph-Clear and ViPER have longer paths on has much longer paths on average, desipte this map being no larger than the simpler ones. This is due to the redundent nature of the clearance process on this map, where pursuers must often backtrack and move together in order to gaurd against recontamiation.

On convex_obstacles, all methods achieve near-complete coverage, although success rates vary. This can be attributed to the many occulde regions, suceptible to recontamination. In this map, we also see much more efficent paths emerge from the ViPER strategy, where its pathes are almost twice as short as the others on average, and the overall timing is much shorter too. This suggests that ViPER's learned policy is able to more efficiently reason about coverage and occlusion dependencies, where other algorithms may opt for a slower, but more methodical approach.

B. Number of Pursuers Comparison

As metioned above, we compare the performance of these strategies with respect to the number of pursuers they must coordiante. Table I deomnstartes this performance on simple_obs, where we see that all methods generally maintain very high coverage, but change in terms of efficiency. with in terms of coverage and path length as the number of pursuers increases.

VII. Open Problems

Overall, much work remains to be done on the topic of motion planning in multi-robot pursuit evasion games. The lack of easily comparable methods has led to difficult to recreate results, and likely hinders real progress on the problem. It would be beneficial for the field in general to standardize problem instances and open source implementations of these algorithms to help model and uncover hidden assumptions in their methodology.

The learning-based and graph-based algorithms tend to do consistently well in covering a large percentage of the area, but often fail to capture the more finegrained details. Meanwhile, the sampling-based algorithms tend to eat too much compute to solve the problem in a reasonable amount of time. The investigation of hybrid methods that utilize the strengths of these paradigms to balance each other's weaknesses could improve the robustness of these methods.

The sampling-based approaches themselves seem convoluted and slow, largely due to the tracking of shadow labels. Improvements in the computational efficiency of these methods would lead to far improved outcomes for these algorithms.

Overall, although interesting, this problem spaces is not particularly developed. Numerous improvements in both procedure and approaches are possible, that would improve the quality of solutions and the applicability of differential games to real-world problems.

References

[1] S. Vajda, "Differential games. a mathematical theory with applications to warfare and pursuit, control and optimization. by rufus isaacs. pp. xxii, 384. 113s. 1965. (wiley)," *The Mathematical Gazette*, vol. 51, no. 375, p. 80–81, 1967.

TABLE I: Performance across various maps with **4** pursuers. Values are reported as $\mu \pm \sigma$ over **30** trials. Coverage $> 99\%$ is considered a success.

Map (size)	Algorithm	Time (s) Plan	Time (s) Exec	Path Length (m)	Max Coverage (%)	Success Rate (%)
`simple_obs` (40×40 m)	**SG-PEG**	**1.35 ± 0.80**	**0.01 ± 0.01**	145.7 ± 92.7	**100.00 ± 0.00**	**100.00**
	VR-SG-PEG	4.75 ± 6.41	0.01 ± 0.00	82.4 ± 52.2	**100.00 ± 0.01**	**100.00**
	Graph-Clear	0.02 ± 0.01	146.42 ± 50.41	99.1 ± 35.1	99.85 ± 0.26	**100.00**
	ViPER	2.14 ± 1.22	0.01 ± 0.00	**73.1 ± 33.8**	99.85 ± 0.15	**100.00**
`ring` (64×64 m)	**SG-PEG**	28.84 ± 16.83	0.01 ± 0.01	87.9 ± 148.7	92.63 ± 10.82	53.33
	VR-SG-PEG	49.32 ± 61.07	0.01 ± 0.01	104.4 ± 93.9	96.69 ± 8.67	83.33
	Graph-Clear	0.02 ± 0.02	27.45 ± 20.54	218.5 ± 132.2	99.62 ± 1.02	90.32
	ViPER	**6.56 ± 4.58**	**0.01 ± 0.01**	**75.7 ± 54.3**	**100.00 ± 0.00**	**100.00**
`H_map` (40×40 m)	**SG-PEG**	139.64 ± 58.88	0.01 ± 0.02	115.2 ± 210.1	75.04 ± 15.93	23.33
	VR-SG-PEG	172.10 ± 33.66	0.00 ± 0.01	**15.4 ± 64.1**	69.77 ± 11.45	6.06
	Graph-Clear	0.05 ± 0.01	100.35 ± 17.46	393.9 ± 76.8	**100.00 ± 0.00**	96.77
	ViPER	**3.22 ± 1.30**	**0.02 ± 0.01**	201.1 ± 79.2	**100.00 ± 0.00**	**100.00**
`convex_obstacles` (40×40 m)	**SG-PEG**	12.15 ± 12.85	0.02 ± 0.01	267.9 ± 164.4	**99.99 ± 0.07**	**100.00**
	VR-SG-PEG	20.46 ± 39.88	0.02 ± 0.01	170.7 ± 100.3	98.78 ± 4.17	90.32
	Graph-Clear	0.04 ± 0.02	63.88 ± 26.76	231.9 ± 57.9	99.00 ± 1.75	77.42
	ViPER	**4.50 ± 2.19**	**0.01 ± 0.00**	**99.1 ± 39.0**	**99.99 ± 0.04**	**100.00**

[2] I. E. Weintraub, M. Pachter, and E. Garcia, "An introduction to pursuit-evasion differential games," 2020. [Online]. Available: https://arxiv.org/abs/2003.05013

[3] F. Liu, X. Dong, Q. Li, and Z. Ren, "Robust multi-agent differential games for general linear systems with model uncertainties∗∗this work was supported by the national natural science foundation of china under grants 61922008, 61973013, 61873011 and 61803014, the innovation zone project under grant 18-163-00-ts-001-001-34, the defense industrial technology development program under grant jcky2019601c106, the beijing natural science foundation under grant 4182035, and the special research project of chinese civil aircraft." *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6691–6696, 2020, 21st IFAC World Congress. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896320303487

[4] Y. Wang, Y. Cao, J. Chiun, S. Koley, M. Pham, and G. A. Sartoretti, "Viper: Visibility-based pursuit-evasion via reinforcement learning," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, Nov. 2025, pp. 4188–4200. [Online]. Available: https://proceedings.mlr.press/v270/wang25k.html

[5] N. M. Stiffler and J. M. O'Kane, "A sampling based algorithm for multi-robot visibility-based pursuit-evasion," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014, pp. 2508–2515.

[6] ——, "Pursuit-evasion with fixed beams," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 4251–4258.

[7] A. Kolling and S. Carpin, "Pursuit-evasion on trees by robot teams," *IEEE Transactions on Robotics*, vol. 26, no. 1, pp. 32–47, 2009.

[8] I. Suzuki and M. Yamashita, "Searching for a mobile intruder in a polygonal region," *SIAM Journal on Computing*, vol. 21, no. 5, pp. 863–888, 1992. [Online]. Available: https://doi.org/10.1137/0221051

[9] L. J. Guibas, J.-C. Latombe, S. M. LaValle, D. Lin, and R. Motwani, "Visibility-based pursuit-evasion in a polygonal environment," *International Journal of Computational Geometry & Applications*, vol. 9, no. 05, pp. 471–494, 1999.

[10] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.

[11] C. M. Clark, "Probabilistic road map sampling strategies for multi-robot motion planning," *Robotics and Autonomous Systems*, vol. 53, no. 3, pp. 244–264, 2005. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889005001326

[12] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of robot motion: theory, algorithms, and implementations*. MIT press, 2005.

[13] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.

[14] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.

[15] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast marching tree," *Int. J. Rob. Res.*, vol. 34, no. 7, p. 883–921, Jun. 2015. [Online]. Available: https://doi.org/10.1177/0278364915577958

[16] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Computer Science Department, Iowa State University, Tech. Rep. TR 98-11, Oct 1998.

[17] T. Olsen, A. M. Tumlin, N. M. Stiffler, and J. M. O'Kane, "A visibility roadmap sampling approach for a multi-robot visibility-based pursuit-evasion problem," *CoRR*, vol. abs/2102.09013, 2021. [Online]. Available: https://arxiv.org/abs/2102.09013

[18] N. M. Stiffler and J. M. O'Kane, "Planning for robust visibility-based pursuit-evasion," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 6641–6648.

[19] ——, "Asymptotically-optimal multi-robot visibility-based pursuit-evasion," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 10 366–10 373.

[20] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[21] Z. Zhou, W. Zhang, J. Ding, H. Huang, D. M. Stipanović, and C. J. Tomlin, "Cooperative pursuit with voronoi partitions," *Automatica*, vol. 72, pp. 64–72, 2016.

[22] H. Huang, W. Zhang, J. Ding, D. M. Stipanović, and C. J. Tomlin, "Guaranteed decentralized pursuit-evasion in the plane with multiple pursuers," in *2011 50th IEEE Conference on Decision and Control and European Control Conference*. IEEE, 2011, pp. 4835–4840.

[23] S. Bhattacharya, G. Paul, and S. Sanyal, "A cops and robber game in multidimensional grids," *Discrete Applied Mathematics*, vol. 158, no. 16, pp. 1745–1751, 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0166218X10002337

[24] C. De Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulić, "Decentralized multi-agent pursuit using deep reinforcement

TABLE II: Performance on **`simple_obs`** for variable numbers of pursuers. Values are reported as $\mu \pm \sigma$ over **30** trials. Coverage $> 99\%$ is considered a success.

# Robots	Algorithm	Time (s)		Path Length (m)	Max Coverage (%)	Success Rate (%)
		Plan	Exec			
2	**SG-PEG**	**38.89 ± 41.32**	**0.01 ± 0.03**	75.7 ± 35.6	**100.00 ± 0.00**	**100.00**
	VR-SG-PEG	50.13 ± 55.39	0.00 ± 0.00	**57.1 ± 21.6**	99.87 ± 0.48	96.67
	Graph-Clear	0.03 ± 0.01	173.81 ± 18.87	87.0 ± 20.1	99.46 ± 0.37	**100.00**
	ViPER	55.88 ± 79.49	0.06 ± 0.08	1025.11 ± 1616.88	99.67 ± 0.26	**100.00**
3	**SG-PEG**	**1.31 ± 0.93**	**0.01 ± 0.00**	131.1 ± 69.7	**100.00 ± 0.01**	**100.00**
	VR-SG-PEG	18.13 ± 28.28	0.00 ± 0.00	**79.8 ± 31.6**	**100.00 ± 0.01**	**100.00**
	Graph-Clear	0.02 ± 0.01	167.37 ± 28.58	101.2 ± 21.1	99.78 ± 0.19	**100.00**
	ViPER	3.38 ± 2.44	0.01 ± 0.00	83.07 ± 42.57	99.81 ± 0.25	**100.00**
4	**SG-PEG**	**1.35 ± 0.80**	**0.01 ± 0.01**	145.7 ± 92.7	**100.00 ± 0.00**	**100.00**
	VR-SG-PEG	4.75 ± 6.41	0.01 ± 0.00	82.4 ± 52.2	**100.00 ± 0.01**	**100.00**
	Graph-Clear	0.02 ± 0.01	146.42 ± 50.41	99.1 ± 35.1	99.85 ± 0.26	**100.00**
	ViPER	2.14 ± 1.22	0.01 ± 0.00	**73.1 ± 33.8**	99.85 ± 0.15	**100.00**
5	**SG-PEG**	3.86 ± 7.35	0.01 ± 0.00	119.2 ± 77.5	99.99 ± 0.07	**100.00**
	VR-SG-PEG	3.98 ± 9.69	0.00 ± 0.00	**63.1 ± 59.3**	**100.00 ± 0.00**	**100.00**
	Graph-Clear	0.02 ± 0.01	142.68 ± 57.46	102.7 ± 40.9	99.86 ± 0.25	**100.00**
	ViPER	**2.13 ± 1.11**	**0.01 ± 0.00**	73.2 ± 42.3	99.90 ± 0.10	**100.00**
6	**SG-PEG**	10.21 ± 12.56	0.00 ± 0.01	84.0 ± 102.3	99.97 ± 0.09	**100.00**
	VR-SG-PEG	2.72 ± 5.52	0.00 ± 0.00	73.1 ± 69.6	**100.00 ± 0.00**	**100.00**
	Graph-Clear	0.02 ± 0.01	142.68 ± 57.46	102.7 ± 40.9	99.86 ± 0.25	**100.00**
	ViPER	**1.96 ± 1.36**	**0.00 ± 0.00**	**65.0 ± 43.1**	99.90 ± 0.11	**100.00**

learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4552–4559, 2021.

[25] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *International conference on machine learning*. PMLR, 2019, pp. 2961–2970.

[26] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. Pmlr, 2018, pp. 1861–1870.